

프롤로그

서버 뼈대에 막히다

오픈이는 이제 막 자바(Java)를 배우기 시작한 학생입니다. 이번 학기 과제인 게시판 웹 서버를 만들기 위해 호기롭게 코드를 짜기 시작했지만, 몇 줄 쓰지도 못하고 벽에 부딪혔습니다.

문제는 게시판 기능이 아니라 그 이전 단계에 있었습니다. 사용자의 요청을 처리하려면, 어떤 주소로 들어왔는지 문자열을 일일이 잘라내어 파악하고 주소마다 실행할 코드를 짚지어주는 작업이 먼저 필요했습니다. 정작 게시글 화면은 하나도 만들지 못했는데, 요청을 분류하기 위한 분기문만 늘어 갔습니다.

과제는 게시판인데, 서버 뼈대만 만들다 학기가 끝나겠네.

다음 날, 오픈이는 같은 수업을 듣는 선배에게 사정을 털어놓았습니다. 선배는 오픈이가 지금까지 쓴 코드를 훑어보더니 말했습니다.

선배 : "이런 걸 처음부터 직접 만드는 사람은 없어요. 웹 서버에 공통으로 필요한 뼈대는 프레임워크에 이미 갖춰져 있거든요. 자바 웹이면 보통 스프링(Spring)을 쓰니까, 그 위에 만들고 싶은 게시판 기능만 구현해 봐요."

선배의 설명을 들은 오픈이는 스프링을 찾아보았습니다. 프레임워크란 기둥과 배관 공사를 마친 건물처럼, 웹 서버 구축에 공통으로 필요한 뼈대를 미리 갖춰 둔 틀을 의미합니다.

클래스에 표식을 붙여 두면 스프링이 그 표식을 읽어 객체로 만들어 두었다가 필요한 곳에 넣어 줍니다. 실행할 메서드를 찾는 일도 스프링이 대신합니다. 개발자가 프레임워크를 호출하는 것이 아니라, 프레임워크가 개발자의 코드를 호출합니다. 이처럼 프로그램의 실행 주도권이 뒤바뀌는 원리를 **제어의 역전 (IoC)** 이라 합니다.

서버의 기본 구조를 스프링에 위임하고 나니, 오픈이가 직접 구현해야 할 부분은 **핵심인 게시판 기능**뿐이었습니다.

게시판을 완성하다

스프링이 자기 코드를 어떻게 찾아 실행하는지 원리를 깨친 오픈이는 본격적으로 스프링을 활용해 보기로 했습니다. 첫 번째 목표는 웹 서비스의 가장 기본이 되는 게시판 만들기였습니다. 새 글을 작성하고, 전체 목록을 조회하며, 작성한 글을 수정하거나 삭제하는 기능을 직접 구현할 계획이었습니다.

하지만 자바 객체와 데이터베이스 테이블은 데이터를 다루는 구조가 다릅니다. 220V 플러그를 110V 콘센트에 바로 꽂을 수 없듯, **자바 객체를 테이블에 그대로 넣을 수는 없습니다**. 이 간극을 메워주는 기술이 **JPA**입니다. 클래스 하나를 테이블 하나에, 객체 하나를 테이블의 한 줄에 대응시키는 방식입니다. 덕분에 오픈이는 게시글 클래스가 어떤 테이블과 짝을 이루는지만 정하면 충분했습니다.

JPA는 데이터베이스와 곧바로 통신하지도 않습니다. 꺼내 온 객체를 잠시 머물게 하는 중간 공간을 두고, 조회나 저장 같은 작업이 모두 이곳을 거치도록 합니다. 이 공간을 **영속성 컨텍스트**라고 부릅니다. 덕분에 같은 글을 두 번 꺼내도 데이터베이스를 다시 조회할 필요가 없습니다.

스프링 프레임워크를 활용한 오픈이는 글을 쓰고, 목록을 보고, 고치고 지우는 게시판의 기본 기능을 모두 구현했습니다. 학기 과제로 기획한 **게시판이 처음으로 동작했습니다**.

실패한 이유를 응답에 담다

게시판의 기본 기능을 완성한 오픈이는 뿌듯한 마음으로 선배에게 화면을 보여줬습니다. 선배는 목록과 상세 조회를 한 번씩 눌러보더니, 주소창 끝에 있는 게시글 번호를 슬쩍 999로 바꿨습니다.

선배 : "없는 번호예요. 이런 예외 상황도 테스트해 봤어요?"

엔터를 치자 응답 창에는 여전히 200 성공이 떴습니다. 하지만 응답 바디는 비어 있었습니다.

게시글이 없는 번호인데 왜 응답이 성공으로 나오지?

선배 : "데이터베이스에서 게시글을 못 찾으면 빈 값으로 응답이 오는데, 이 빈 값을 그대로 응답에 담아버리니 성공으로 응답이 뜨는 거예요. 만약 없는 게시글을 수정이나 삭제하려고 한다면 오류가 발생하겠죠. 서버에서는 이런 예외(Exception)가 발생했을 경우를 위해 미리 예외 처리를 해야 해요. 그래야 어떤 사유로 요청이 실패했는지 클라이언트에게 명확히 알려줄 수 있어요."

선배의 말대로 오픈이는 조회 결과를 곧바로 응답에 담지 않았습니다. 대신 게시글이 실제로 존재하는지 먼저 확인하고, 없다면 곧바로 예외를 발생시키도록 흐름을 바꿨습니다. 이렇게 발생한 예외는 **전역 예외 처리기**가 한곳에서 모아 받아 알맞은 상태 코드와 사유로 바꿔 내보냅니다. 코드를 고치고 없는 번호인 999번을 다시 조회하자, 이번에는 에러 상태 코드와 함께 게시글을 찾을 수 없다는 명확한 사유가 돌아왔습니다.

응답에 담아 보내던 데이터도 함께 수정했습니다. 테이블 구조와 같은 객체를 그대로 응답하던 구조는 불필요한 정보들이 함께 응답되어 보안 등의 문제가 발생할 수 있었습니다. 이를 해결하기 위해 오픈이는 필요한 컬럼만 따로 담은 **DTO**라는 객체를 추가했습니다.

이제 프로젝트는 예외가 발생했을 때 필요한 응답을 보내는 서버로 바뀌었습니다.

요청한 사람을 식별하다

게시글 기능이 완성되었으니, 다음은 회원을 추가해 인증 게시판을 만들 순서였습니다. 오픈이는 회원 테이블을 만들고 회원가입과 로그인 기능을 구상했습니다. 하지만 로그인이 성공한 이후가 문제였습니다. 게시글의 작성자와 요청을 보낸 사용자가 같은지를 어떻게 구분할지 떠오르지 않았습니다.

오픈이는 선배를 찾아갔습니다.

오픈이 : "선배님, 로그인은 아이디와 비밀번호를 확인하면 되는데, 그 이후에 들어오는 요청은 서버가 누가 보냈는지 어떻게 아나요?"

선배 : "서버는 기본적으로 요청만 처리할 뿐, 누가 보냈는지는 기억하지 않아요. 그래서 로그인에 성공한 사용자에게 인증 수단을 줘야 해요. 클라이언트가 이후 요청을 보낼 때마다 이걸 함께 보내면, 서버는 그걸 보고 누군지 식별하는 거예요. 그래야 글을 수정하거나 삭제할 때 작성자 본인이 맞는지 검증할 수 있거든요."

오픈이가 찾은 인증 수단은 **JWT**였습니다. 로그인에 성공하면 회원 정보를 담은 JWT를 발급하고, 이후 요청은 필터를 거쳐 서명을 검증하도록 만들었습니다. 서버가 로그인 상태를 따로 보관하지 않아도 요청마다 누가 보냈는지 확인할 수 있습니다.

여기까지가 요청을 보낸 사람의 신원을 확인하는 **인증**입니다. 사원증을 찍고 게이트를 지나는 것이 인증이라면, 자료실처럼 권한을 받은 사람만 들어갈 수 있는 것은 **인가**입니다. 오픈이는 게시글에 작성자를 연결해 두고, 수정과 삭제 요청이 들어오면 작성자와 요청자가 같은지 비교하게 했습니다. 작성자와

요청자가 다르면 권한이 없다는 응답이 돌아옵니다. **자기가 쓴 글만 고치고 지울 수 있는 게시판**이 되었습니다.

게시글과 댓글을 연결하다

오픈이는 인증 게시판에 이어 댓글 기능을 추가하기로 했습니다. 게시글을 조회할 때 여러 개의 댓글을 함께 보여줘야 하고, 게시글과 댓글은 각각 작성자 정보를 가지고 있었습니다.

댓글 테이블을 설계하던 오픈이는 머릿속이 복잡해졌습니다.

게시글, 게시글 작성자, 여러 개의 댓글, 그리고 댓글 작성자들까지. 이 많은 데이터를 한 화면에 다 보여주려면 어떻게 가져와야 하지?

오픈이는 선배를 찾아갔습니다.

오픈이 : "선배님, 게시글 상세 화면에 댓글까지 추가되니까 데이터를 어떻게 가져와야 할지 막막하네요."

선배 : "**JPA의 양방향 매핑을 쓰는 거예요.** 데이터베이스 테이블은 외래 키 하나로 양방향 조인이 가능하지만, 자바 객체는 참조하는 쪽으로만 갈 수 있는 일방통행이거든요. 그래서 양방향 매핑으로 게시글 객체 쪽에서도 댓글을 참조할 수 있게 열어주는 거예요. 이렇게 해두면 JPA에서 조인을 통해 한 번에 데이터를 가져올 수 있어요."

오픈이는 게시글 객체에 댓글 목록을 참조하는 필드를 추가했습니다. 그러자 댓글에서 게시글로만 가던 참조가 양쪽으로 열렸습니다. 연관된 데이터는 평소에 가져오지 않다가 실제로 쓸 때 조회하고, 게시글과 댓글을 한 화면에 함께 보여줄 때만 조인으로 묶어 한 번에 가져옵니다. 댓글이 없는 게시글이 결과에서 빠지지 않도록 외부 조인을 사용했습니다.

게시글 하나를 조회하자 작성자와 댓글 목록이 한 응답에 담겨 왔습니다. 학기 초에는 주소를 자르는 분기문 앞에서 멈춰 있던 과제가, **댓글까지 오가는 게시판**이 되었습니다.